

AEMaaCS Deep-Dive Volume – Authors (Sites Authors & Content Owners)

AEMaaCS Deep-Dive Volume

For Authors (Sites Authors & Content Owners)

This volume focuses on *how to author well* on AEMaaCS, with concrete tips for Core Components, templates, assets, and day-to-day workflows.

1. Author Mindset in AEMaaCS

- Your job is **structure + story**, not just “filling pages”.
- Think in **reusable building blocks**:
 - Templates, Core Components, Content Fragments, Experience Fragments.
- Aim for **consistency and speed**:
 - Reuse patterns and fragments; avoid copy-paste between pages.

References

- [Authoring overview](#)
 - [Components \(Page Editor\)](#)
-

2. Page Editor, Templates, and Layout

2.1 Page Editor basics

Key concepts

- **Components** – your building blocks (Text, Image, Teaser, Carousel, etc.).
- **Policies and templates**:
 - Template = skeleton of the page: allowed components, fixed areas (header/nav/footer).
 - Policy = design settings for components (styles, options).
- **Side panel**:
 - Components browser, Assets browser, Content tree.

Tips

- Use the **Content Tree** to:
 - Debug layout issues (see which components wrap which).
 - Reorder components more precisely than drag-and-drop.
- Make **small, frequent saves** with meaningful page titles and descriptions.

References

- [Introduction to authoring and publishing \(WKND Quick Site\)](#)
-

3. Core Components – Practical Usage Tips

3.1 General patterns

- Prefer **Core Components** over custom ones when possible:
 - They are responsive, accessible, SEO-friendly, and tested.
- Look for the **"Styles"** dropdown:
 - Many components have multiple visual variants under the same component (e.g., Teaser vs Hero).
- Rely on **policies**:
 - If you don't see an option you expect, ask a template author to expose it via the component's policy.

References

- [Author with Core Components](#)
- [Core Components Introduction](#)
- [Understanding Core Components](#)

3.2 Text & Title components

Best practices

- Use **Title** component for page headings:
 - Let it render <h1>, <h2>, etc. Use one <h1> per page.
- Use **Text** component for rich text:
 - Lean on paragraph and heading styles; avoid manually changing fonts/colors inline.
- For **SEO-friendly content**:
 - Use headings to structure the page logically.

- Avoid using Title component just for visual styling; it affects your heading structure.

Tricks

- If your site uses a “lead-in” style (intro text), ask for a **style variation** on Text instead of using extra components.

3.3 Image component

Best practices

- Always fill **Alt Text** thoughtfully:
 - Think “how would I describe this image to someone who cannot see it?”
- Use **Smart Crops / responsive options**:
 - If enabled, preview how images look on mobile.
- Use **DAM renditions** where appropriate:
 - Don’t upload separate copies of the same image just to crop it differently.

Tricks

- If a hero image is slow to load:
 - Check if you’re using the **right rendition** and not a massive original; talk to your DAM librarian if unsure.

3.4 Teaser, List, Carousel, and Navigation

Teaser

- Ideal for promotional blocks:
 - Use teaser **actions (links)** consistently; avoid empty teaser links.
- Reuse imagery from DAM to maintain consistency.

List

- Use **content-driven lists** instead of manually creating grids of links:
 - Lists that pull from a folder or search criteria are more maintainable.

Carousel / Tabs / Accordion

- Don’t overuse:
 - These widgets hide content; use them for structured journeys or FAQs, not everything.
- Keep slides/panels focused and short.

Navigation / Breadcrumb

- Navigation is usually template-managed:
 - Avoid trying to “fake” navigation with manual link lists; use real navigation components configured at the template/site level.

References

- [Components \(Page Editor\)](#)
 - [Building Effective Web Pages in AEM Sites](#)
-

4. Layout Container, Grid, and Styles

4.1 Layout Container & responsive grid

Patterns

- Use **Layout Container** to:
 - Create multi-column layouts.
 - Control how components stack on mobile/desktop.
- Check the **responsive breakpoints**:
 - Use the device preview icons to see mobile/tablet views.

Tips

- Avoid deeply nested layout containers:
 - A common anti-pattern is container-in-container-in-container; it becomes hard to manage.
- Prefer **template-level layout decisions**:
 - If many pages share the same layout, push it into the template rather than recreating it each time.

4.2 Style System

- Look for **Styles** dropdown in component toolbar:
 - e.g., “Hero”, “Card with Image”, “Muted”.
- Use styles instead of:
 - Ad-hoc inline formatting (fonts, colors, manual spacing).

Trick

- If you find you need to copy a component just to change its visual treatment, you probably want a **new style** instead.

Reference

- [Author with Core Components – Component Styling & Policies](#)
-

5. Reusable Content: CFs, XFs, and Fragments

5.1 Content Fragments (CFs)

Use CFs when:

- Content is **structured and reused** across pages/channels:
 - e.g., job postings, event details, product specs.
- You need **headless delivery** via GraphQL to other apps.

Best practices for authors

- Treat CF Models like a **schema**:
 - Don't "work around" the model by stuffing extra info into free-text fields.
- Use CF variants for:
 - Different locales, channels, or representations (short vs long description).

References

- [AEM GraphQL API for use with Content Fragments](#)
- [Content Fragments - Setup](#)

5.2 Experience Fragments (XFs)

Use XFs when:

- You need **reusable visual sections**:
 - e.g., hero banners, promos shared across many pages/sites.
- You might export them to other channels (email, other sites, etc.).

Best practices

- Create **named, purposeful fragments**:
 - "Global – Footer v3 – Dark", not "footer-test".

6. Assets Usage for Authors

6.1 Finding and using assets

Patterns

- Use **search and filters**:
 - Filter by tags, file type, orientation, and metadata rather than browsing deep folders.
- Respect **permissions and usage rights**:
 - If your org uses rights metadata, check license/expiration before using.

Tips

- Favorite commonly used assets or use **Collections** (where enabled) to group assets for campaigns.

References

- [Organize your digital assets](#)
- [AEM Assets videos and tutorials overview](#)

6.2 Performance-conscious authoring

- Avoid:
 - Uploading **huge original files** and using them directly on pages.
- Prefer:
 - Standard renditions configured by DAM (web, thumbnail, social, etc.).
- If pages feel slow:
 - Check for many unoptimized images or embedded third-party content.

7. Workflows, Versioning, and Scheduling

7.1 Versioning and approvals

Best practices

- Use **page versions** before major updates:
 - Especially for high-traffic pages.

- Use **approval workflows** where configured:
 - Avoid publishing critical pages without at least another pair of eyes.

7.2 Scheduling and launches

- For time-based content:
 - Use **"Schedule Activation/Deactivation"** in page properties.
 - For broader site refreshes:
 - Use **Launches** (if your program relies on them) for planning big changes.
-

8. Authoring for SEO, Accessibility, and Performance

8.1 SEO-friendly authoring

Patterns

- Use semantic structure:
 - `<h1>` for page title (once), `<h2>` for sections, `<h3>` for subsections.
- Write meaningful:
 - Page titles, meta descriptions, and internal link labels.
- Avoid:
 - "Click here" links; instead use descriptive anchor text.

8.2 Accessibility

Tips

- Always fill **Alt Text**.
 - Avoid putting critical information only in:
 - Images or color cues; provide text equivalents.
 - Use **lists and headings** rather than manual formatting.
-

9. Common Authoring Anti-Patterns

- **Copy-paste pages** for every campaign:
 - Prefer templates, fragments, and lists.
- Overusing **Carousels/Accordions** to hide lots of copy:
 - Better to simplify or split content into multiple pages.

- Inline styling in Text component:
 - Leads to inconsistent look and harder redesigns.
-

10. Example Authoring Flows

10.1 Building a campaign landing page

1. Start from the **correct template** (campaign/landing).
2. Add **hero** using Teaser or Hero style with a DAM asset.
3. Add **Text** and **Image** components for the body.
4. Use **List** to pull in related articles or events.
5. Save, **Preview** on mobile, then schedule activation.

10.2 Publishing a new event

1. Create **Content Fragment** using the Event model.
 2. Fill structured fields (title, date, location, description).
 3. Place CF on event listing page via a **CF-based component**.
 4. Verify that GraphQL (if used) surfaces the event in external apps.
-

References (Authors Volume)

- [Overview of Authoring in AEM Sites](#)
- [Components \(Page Editor\)](#)
- [Author with Core Components](#)
- [Core Components Introduction](#)
- [Understanding Core Components](#)
- [Building Effective Web Pages in AEM Sites](#)
- [AEM Assets videos and tutorials overview](#)
- [Organize your digital assets](#)